

Study Group Module

Mini Software Requirements Specification and implementation plan

Purpose Deliver a secure, collaborative study-space experience for verified college users.	Release scope P0: collaboration core. P1: storage, search, audit, reliability, chat.
--	--

Document control

Service	Study Group Service
Base path	/api/v1/study-groups
Audience	Product, backend, web, QA, platform
Status	Implementation-ready draft
Date	22 August 2026

Success looks like

Verified students can create and join study groups, collaborate through role-governed discussions and versioned notes, and have every important change protected by tenancy, authorization, validation, and an auditable trail.

1. Product scope

The module provides tenant-isolated study groups for SUBJECT, EXAM, PROJECT, and ASSIGNMENT collaboration. A group has members and roles, discussion threads with replies, shared notes with immutable version history, and later extensions for files, timetables, search, audit, and chat.

In scope

Groups	CRUD; public/private policy; group type; owner	Discovery/search filters and ranking
Membership	Join/leave; invitations; owner/admin/member roles	Idempotency and richer eventing
Collaboration	Threads, replies, shared notes, note versions	Files, timetable, chat bridge
Platform controls	Tenant isolation, verified user, feature flag, RBAC	Audit viewer, optimistic concurrency

Out of scope for this release

Real-time message transport, file binary handling, calendar synchronization, full-text indexing infrastructure, notification delivery, and cross-tenant sharing. P1 endpoints may be implemented behind flags while their platform dependencies are completed.

Primary user roles

OWNER	Full group control; transfer/delegate ownership; delete group; manage roles and membership.
ADMIN	Manage members within policy; moderate discussions; manage notes, timetable, and files.
MEMBER	Read group content; join/leave where allowed; create and edit own discussion/reply content; create/edit shared notes subject to policy.

2. Functional requirements

All protected requests must authenticate the caller, resolve an active tenant/college, require verified-user status, check the Study Groups feature flag, and return only tenant-scoped data.

Group and membership behaviour

SG-01	Create group	Creates an OWNER membership for caller and emits GroupCreated. Required: name, type; optional description, visibility, capacity.
SG-02	List/detail/update/delete	Only tenant-owned groups appear. Update/delete require OWNER; mutation supports If-Match when enabled.
SG-03	Join/leave	Join respects visibility/capacity; leave removes caller unless owner transfer/delete rule applies. Duplicate requests are safe with Idempotency-Key.
SG-04	Membership admin	OWNER may invite/promote/demote/remove; ADMIN scope is limited by policy; owner cannot be removed without transfer.

Collaboration behaviour

SG-05	Discussions and replies	Members can create/read. Authors edit/delete own content; ADMIN/OWNER may moderate. Soft delete preserves audit context.
SG-06	Shared notes	Members create and edit notes. Each successful update stores a new immutable version and returns current version/ETag.
SG-07	Note history	Members can list versions and retrieve a specific version; historic content is read-only.
SG-08	P1 resources	Files use upload/download URLs and server-side metadata validation; timetable stores revisions; group and global search honor tenancy and membership visibility.

P1 integration and observability

Chat endpoint provisions or retrieves the mapped conversation through the existing Chat Service. Audit endpoint returns authorized, redacted event history. Domain events are emitted using an outbox to avoid dual-write loss.

3. API contract and service rules

Route groups are implemented under apps/api/modules/study-groups using handler, service, repo, router, and lib layers. Expose the router from the API composition root.

Groups	POST/GET /study-groups; GET/PATCH/DELETE /:groupId; POST /:groupId/join, /leave	Authenticated; owner for mutation/delete; member where applicable
Members	GET /:groupId/members; POST invitations; PATCH/DELETE members/:userId	Member to list; owner/admin per role policy
Discussions	CRUD /discussions; CRUD nested /replies	Member read/create; author or moderator edit/delete
Notes	CRUD /notes; GET version list/detail	Member access; shared-note edit policy
Files & timetable	Upload URL, file metadata/list/get/download/delete; timetable CRUD/history	Member read; admin/owner write by policy
Search, chat, audit	Global/group search; chat create/get; group audit	Tenant visibility; member for group search/chat; owner/admin for audit
Health	GET /health and /ready	Unauthenticated platform checks

Request and response conventions

Headers	Authorization: Bearer token on protected routes; Idempotency-Key on state-changing retrievable operations; If-Match for version-controlled updates.
Pagination	Cursor pagination with stable sort (createdAt DESC unless documented otherwise). Return items, nextCursor, and total only where inexpensive.
Errors	Use one error envelope: code, message, requestId, details. Map 400 validation, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict, 412 version mismatch, 422 domain rule, 429 rate limit.
Safety	Never leak existence across tenants; redact actor PII in audit results; validate IDs, payload size, HTML/Markdown policy, file type/size, and query length.

4. Data model and architecture

Use Prisma in @lumina/db. All business tables include tenantId, stable IDs, createdAt, updatedAt, and appropriate tenant-aware indexes. Services own transactions; repositories do persistence only.

StudyGroup	id, tenantId, name, description, type, visibility, capacity, ownerId, version, deletedAt. 1:N memberships/discussions/notes/files/timetable revisions/audit.
StudyGroupMember	groupId + userId unique; role (OWNER/ADMIN/MEMBER), status, joinedAt, invitedBy. Index tenant/group/role.
Discussion / Reply	groupId, authorId, body, version, deletedAt; Reply belongs to Discussion.
StudyNote / StudyNoteVersion	Note current body/title/version; versions store immutable snapshot, editedBy, version number, timestamp. Unique noteId + version.
P1: FileAsset / TimetableRevision	File metadata/object key/checksum/status; timetable snapshot with monotonically increasing revision.
AuditEvent / OutboxEvent	Actor, action, entity type/id, before/after safe metadata, requestId; outbox payload/status for reliable domain events.

Control flow for every protected mutation

Authenticate -> resolve tenant + verified status -> feature flag -> load resource tenant-scoped -> membership and role check -> validate -> idempotency lookup -> transaction (state change + version/audit/outbox) -> cache/event dispatch -> safe response.

Key policies to settle before build

Visibility	Private by default. Join is allowed only for discoverable/open groups; invitation can add members to private groups.
Capacity	Optional; join returns 422 when full.
Note conflicts	Require If-Match for PATCH; return 412 plus current version metadata on mismatch.
Deletion	Soft-delete discussion/replies/groups; hard delete only through retention job. Group delete cascades access removal but preserves audit.
Owner exit	Block leaving as sole owner; require transfer or delete first.

5. Roadmap and delivery plan

0. Foundations	Schema migrations; feature flag; tenant/auth middleware; shared validation/errors; OpenAPI contract; test fixtures.	Migration applies cleanly; all requests tenant-scoped; contract review complete.
1. P0 groups & RBAC	Group CRUD, memberships, invitations, join/leave, role guard, audit skeleton.	Permission matrix and edge-case tests pass; owner protection verified.
2. P0 collaboration	Discussions/replies; notes + immutable versions; optimistic concurrency.	Author/moderator rules and version retrieval are covered by integration tests.
3. P1 platform	Signed file workflow + validation, timetable revisions, search, audit feed, outbox events, idempotency.	Storage security tests; no duplicate mutations; audit/event completeness sampled.
4. Chat & release	Chat Service adapter; web integration; observability dashboards; load/security checks; rollout flag.	Staged tenant rollout meets latency/error targets and has rollback path.

Implementation work breakdown

Backend: Prisma models/migrations; validators in @lumina/validators; module router/service/repo; authorization library; storage/chat adapters; outbox worker.

Web: group directory/detail, member management, threads, note editor/history, error/conflict states; P1 files/timetable/search.

QA: route contract tests, RBAC matrix, tenant-isolation tests, concurrency/idempotency tests, storage abuse tests, accessibility and smoke tests.

Quality gates

Security	No cross-tenant reads/writes; authorization tests for each endpoint; signed URLs scoped and short lived.
Reliability	Transactions include audit/outbox; retry-safe idempotent routes; 412 behavior tested.
Performance	Cursor pagination; composite indexes for tenant/group/createdAt; load-test list/search paths.
Observability	requestId, structured action logs, metrics for membership/content mutations, audit/outbox failure alerts.

Risks and mitigations

Role complexity / privilege escalation	Centralize policy checks and test table-driven role matrix.
Lost updates to notes/timetable	Enforce version checks and retain immutable revisions.
File abuse or data leakage	Direct-to-object-store signed URLs, MIME/size/checksum validation, malware scanning before availability.
Chat dependency delays	Adapter interface plus feature flag; module stays usable without chat.